

UNIVERSIDAD DE MANIZALES

Tecnología en Desarrollo de Software para Negocios Digitales
Seguridad de Aplicaciones

Universidad de Manizales

Sergio Pinilla Valencia

Pablo Nieto Pérez

NegocioSeguro (Compra Segura)

Tarea 4. owasp Top 10 y Vectores de Inyección sql

15 de Octubre de 2026

Introducción

Para esta tarea volví a agarrar la aplicación negocio seguro que es la misma compra segura que veníamos revisando en las clases pasadasy esta vez la miré con el owasp Top 10 del 2021, enfocándome en la categoría de Inyección, la A03y dentro de esa sobre todo en la inyección sql

Lo que hice fue montar el proyecto en mi xampp local y seguir la guía de la práctic, primero reproduje tres ataques de inyección en mi propio equipo y después me puse a leer el código de otros dos archivos para buscar más puntos vulnerables

El informe sigue la plantilla de la materia: primero los objetivos y la justificacióny luego el desarrollo, que es la parte más larga, ahí va el contexto de owasp, las tres inyecciones que reproduje, los dos puntos que encontré leyendo el código, la tabla con los cinco hallazgos y la forma de corregirlos

Objetivos

Objetivo general

Analizar la aplicación NegocioSeguro frente a la categoría A03 (Inyección) del owasp Top 10:2021, demostrando en un entorno local controlado cómo funciona la inyección sql y el riesgo que representa para el negocio.

Objetivos específicos

1. Reproducir en el entorno local la inyección basada en errores y la basada en union sobre buscar_producto.phpy la inyección ciega booleana sobre login.php.
2. Identificar en el código fuente de registro.php y pedido.php la línea exacta donde existen puntos de inyección adicionales.
3. Consolidar los cinco hallazgos en una tabla con el archivo, la línea, la variante, el payload, el resultado y el impacto de seguridad.
4. Proponer la corrección de cada punto vulnerable con consultas preparadas y controles complementarios.
5. Argumentar cuál de los vectores representa el mayor riesgo para el negocio.

Justificación

NegocioSeguro maneja justo los datos que un atacante quisiera: los usuarios con sus contraseñas, que además están guardadas en texto planoy los pedidos con su precio y su total. En las tareas anteriores ya habíamos visto que el login se podía saltar con ' or '1'='1, entonces esta vez quise entender bien cómo funciona la inyección sql por dentro y hasta dónde llega el daño.

Me apoyé en el owasp Top 10 porque es la referencia con la que se audita la seguridad de aplicaciones weby la Inyección, la A03, es la que más pesa en esta appya que todas sus consultas a la base de datos se arman pegando lo que escribe el usuario dentro del sql. Revisando archivo por archivo puedo señalar la línea exacta del fallo y proponer una corrección que se aplique directo en el código.

Desarrollo

Contexto y metodología

Aplicación auditada: NegocioSeguro (Compra Segura), repositorio https://github.com/pablowib/CompraSegura, commit b24a5de.

Entorno: XAMPP (Apache, php y mariadb/mysql) en http://localhost/compraseguras/, con phpMyAdmin para revisar la base de datos.

Metodología: primero preparé el entorno para que se vieran los errores de la base de datos, después reproduje las tres variantes de inyección pedidas en la guía, luego revisé por código registro.php y pedido.phpy por último consolidé todo en la tabla de hallazgos y propuse la corrección.

Qué es owasp y el Top 10

owasp (Open Worldwide Application Security Project) es una organización sin ánimo de lucro que publica material gratis sobre seguridad en el software. Lo más conocido que tienen es el owasp Top 10, que es una lista con los diez riesgos más graves que se ven en las aplicaciones web. La van actualizando cada cierto tiempoy la que está vigente es la del 2021; la lectura de clase de Fortinet todavía trae la anterior, la de 2017. Estos son los diez de la edición 2021:

Código	Categoría	Qué es
A01	Pérdida de control de acceso	El usuario llega a cosas que no le corresponden.
A02	Fallos criptográficos	Datos sensibles sin cifrar o mal protegidos (claves en texto plano, por ejemplo).
A03	Inyección	Datos del usuario se terminan ejecutando como código o consulta (sqli, xss, comandos).
A04	Diseño inseguro	El fallo está en el diseño, no solo en el código; faltan controles desde el principio.
A05	Configuración incorrecta	Permisos de más, errores visibles, valores por defecto inseguros.
A06	Componentes vulnerables	Librerías o servidores viejos con fallos ya conocidos.
A07	Fallos de identificación y autenticación	Login flojo, sin control de intentos, sesiones mal manejadas.
A08	Fallos de integridad	Se confía en datos o código sin verificar que no los cambiaron.
A09	Fallos de registro y monitoreo	No se guardan ni se detectan los ataques.

A10	ssrf	Al servidor se le obliga a pedir recursos que no debería.

Compra Segura cae en varias de estas, pero la que más pesa es la A03, Inyección, que es sobre la que trabaja esta tarea.

Los vectores de inyección sql

La inyección sql pasa cuando lo que escribe el usuario se pega directo dentro de una consulta sql sin separarlo de la consulta misma, entonces la base de datos termina leyendo parte de ese texto como si fueran órdenes y no como un simple dato. Los tipos que hay son básicamente estos:

- En banda: el resultado del ataque se ve en la misma respuesta de la página. Puede ser basada en error (se provoca un error de la base de datos para que el mensaje suelte información de adentro, como la versión o las tablas), basada en union (se pega un union select para leer datos de otras tablas) o por tautología (se mete una condición que siempre da verdadera, típico para saltarse un login).
- Ciega (blind): no se ve el dato directamente, se va deduciendo por cómo reacciona la página. Puede ser booleana (la respuesta cambia según si la condición es verdadera o falsa) o por tiempo (se mide cuánto tarda en responder, con sleep y parecidos).
- Fuera de banda: el dato se saca por otro canal (dns o http). Es raro y depende de cómo esté configurado el servidor.

Al final el origen siempre es el mismo, que es mezclar el código con los datosy la solución también es siempre la misma: usar consultas preparadas con parámetros, que separan la estructura de la consulta de los datos, además de validar las entradas y no dar más permisos de los necesarios en la base de datos.

Preparación del entorno

Para que las inyecciones basadas en error muestren el mensaje de la base de datos, toca activar que se vean los errores. En conexion.php, apenas abre el php, agregué estas dos líneas:

ini_set('display_errors', 1);

error_reporting(e_all);

Aclaro que esto es solo para la práctica en local. En un sitio real mostrarle los errores al usuario ya es de por sí un fallo de seguridad (A05), porque le estás dando información de la aplicación. Lo correcto allá sería mandarlos a un archivo de log y no imprimirlos en pantalla.

Inyecciones demostradas

Inyección 1: basada en error (buscar_producto.php, línea 5)

La consulta que tiene el problema es esta:

```
$sql = "select * from productos where nombre like '%$busqueda%'";
```

Lo que escribe el usuario en el buscador entra sin ningún filtro, entonces aprovechando eso forcé un error de una función de xml de la base de datos para que el mensaje de error me soltara un dato de adentro, en este caso la versión del motor.

- Campo: el buscador de productos.
- Lo que ingresé: ' and extractvalue(1,concat(0x7e,(select version()))) -- -
- Qué pasó: la página no mostró productos, sino el error de la base de datos «xpath syntax error», con la versión del motor justo después del símbolo ~. O sea, me dijo qué versión está corriendo.
- Impacto: Confidencialidad. Todavía no robé datos, pero ya tengo información del sistema que me sirve para armar ataques más dirigidos.

Inyección 2: basada en union (buscar_producto.php, línea 5)

Es la misma consulta de arriba. La tabla productos tiene 4 columnas (id, nombre, precio y descripcion), entonces le pegué un union select también de 4 columnas pero apuntando a la tabla de usuarios.

- Campo: el buscador de productos.
- Lo que ingresé: inexistente' union select id, usuario, clave, 'robado' from usuarios -- -
- Qué pasó: como busqué un producto que no existe, la primera consulta no trae nada, pero el union engancha la segunda. Como la página imprime el nombre y el precio de cada fila, en pantalla salieron los usuarios con sus contraseñas en texto plano en ese lugar: «admin - $admin123» y «jperez - $miclave2024».
- Impacto: Confidencialidad. Esto ya es sacar credenciales directamente de otra tabla. Peor todavía porque las claves están sin cifrar (A02).

Inyección 3: ciega booleana (login.php, línea 8)

La consulta vulnerable del login es:

```
$sql = "select * from usuarios where usuario = '$usuario' and clave = '$clave'";
```

Acá la página no muestra ni datos ni errores, pero sí cambia la respuesta según si lo que pregunto es verdadero o falso, entonces la idea es ir adivinando la clave del usuario admin letra por letra.

- Campo: el de usuario en el login (la clave no importa porque la corto con el comentario -- -).
- Caso verdadero: admin' and substring(clave,1,1)='a' -- -
- Caso falso: admin' and substring(clave,1,1)='z' -- -
- Qué pasó: con la 'a' el sistema me dejó entrar y mostró «Bienvenido, admin. Acceso concedido.», o sea que la clave sí empieza por 'a'. Con la 'z' me dijo «Usuario o clave incorrectos.». Nunca vi la clave, pero por la respuesta fui confirmando cada carácter. Repitiendo eso se saca la clave completa.
- Impacto: Confidencialidad. Saco las credenciales pero de forma silenciosa. También toca A07 porque es un fallo del login.

Puntos de inyección identificados en el código

registro.php, línea 6

```
4   $usuario = $_post['usuario'];
5   $clave   = $_post['clave'];
6   $sql = "insert into usuarios (usuario, clave) values ('$usuario', '$clave')";
```

El usuario y la clave llegan del formulario y se pegan tal cual dentro del insert, en la línea 6. Con solo meter una comilla simple en cualquiera de los dos campos ya se rompe la consultay con algo más armado se podría alterar lo que se guarda. Y aparte de eso, la clave queda guardada en texto plano (A02).

- Línea vulnerable: 6, en los campos usuario y clave.
- Impacto: Integridad. Se pueden dañar o modificar los datos de la tabla usuarios.

pedido.php, línea 10

```
4    $producto_id = $_post['producto_id'];
5    $precio      = $_post['precio'];
6    $cantidad    = $_post['cantidad'];
7    $usuario_id  = $_session['usuario_id'] ?? 0;
8    $total = $precio * $cantidad;
10   $sql = "insert into pedidos (usuario_id, producto_id, cantidad, total) values
('$usuario_id','$producto_id','$cantidad','$total')";
```

Igual que en registro, el producto_id y la cantidad entran sin filtrar al insert, en la línea 10. Pero este archivo además tiene dos problemas de diseño que una consulta preparada por sí sola no arregla:

1. El precio viene de un campo oculto del formulario (<input type="hidden" name="precio" value="3000">). El usuario puede cambiar ese valor y pagar lo que quiera (A04 y A08).
2. Con $_session['usuario_id'] ?? 0 se pueden registrar pedidos sin haber iniciado sesión (A01).

- Línea vulnerable: 10, en los campos producto_id y cantidad.
- Impacto: Integridad (se manipulan los datos del pedido) y además afecta al negocio porque se puede pagar menos de lo real.

Tabla consolidada de hallazgos

La tabla reúne los cinco hallazgos: tres demostrados en el entorno local y dos identificados en el código.

#	Archivo	Línea / campo	Variante	Payload usado	Resultado observado	Impacto
1	buscar_producto.php	Línea 5, campo de búsqueda	Basada en error	' and extractvalue(1,concat(0x7e,(select version()))) -- -	Error «xpath syntax error» con la versión del motor después del ~	Confidencialidad (reconocimiento)

2	buscar_producto.php	Línea 5, campo de búsqueda	union	inexistente' union select id, usuario, clave, 'robado' from usuarios -- -	Salen «admin - $admin123» y «jperez - $miclave2024» en lugar de los productos	Confidencialidad (fuga de credenciales)
3	login.php	Línea 8, campo usuario	Ciega booleana	admin' and substring(clave,1,1)='a' -- - (y con 'z' para el caso falso)	Con 'a' deja entrar, con 'z' no; así se saca la clave letra por letra	Confidencialidad (extracción silenciosa)
4	registro.php	Línea 6, usuario y clave en el insert	Identificada en código	No aplica (análisis de código)	El post se concatena directo en el insert; se pueden alterar los datos	Integridad
5	pedido.php	Línea 10, producto_id y cantidad en el insert	Identificada en código	No aplica (análisis de código)	El post entra directo al insert, más el precio puesto por el cliente y pedidos sin sesión	Integridad / negocio

Corrección propuesta

Lo importante es no concatenar nunca lo que escribe el usuario dentro del sql y usar consultas preparadas con parámetros (prepare y bind_param). Así quedarían los cuatro archivos.

registro.php, con consulta preparada y la clave cifrada:

```
$hash = password_hash($_post['clave'], password_default);
$stmt = $conn->prepare("insert into usuarios (usuario, clave) values (?, ?)");
$stmt->bind_param("ss", $_post['usuario'], $hash);
$stmt->execute();
```

login.php, con consulta preparada y verificando el hash, sin comparar la clave dentro del sql:

```
$stmt = $conn->prepare("select id, usuario, clave from usuarios where usuario = ?");
$stmt->bind_param("s", $_post['usuario']);
$stmt->execute();
$fila = $stmt->get_result()->fetch_assoc();
if ($fila && password_verify($_post['clave'], $fila['clave'])) {
    $_session['usuario_id'] = $fila['id'];
    $_session['usuario']    = $fila['usuario'];
}
```

buscar_producto.php, poniendo los % dentro del parámetro y no en la cadena:

```
$stmt = $conn->prepare("select nombre, precio from productos where nombre like ?");
$like = '%' . $_get['q'] . '%';
$stmt->bind_param("s", $like);
$stmt->execute();
```

pedido.php, leyendo el precio de la base de datos, con consulta preparada y exigiendo sesión:

```
if (empty($_session['usuario_id'])) { header('Location: login.php'); exit; }
$stmt = $conn->prepare("select precio from productos where id = ?");
$stmt->bind_param("i", $_post['producto_id']);
$stmt->execute();
$precio = $stmt->get_result()->fetch_assoc()['precio'];   // sale de la bd, no del formulario
$cantidad = (int) $_post['cantidad'];
$total    = $precio * $cantidad;
$ins = $conn->prepare("insert into pedidos (usuario_id, producto_id, cantidad, total) values (?,?,?,?)");
$ins->bind_param("iiid", $_session['usuario_id'], $_post['producto_id'], $cantidad, $total);
$ins->execute();
```

Aparte de las consultas preparadas, hay otras cosas que conviene arreglar:

- Guardar las claves con password_hash y password_verify (eso cierra el A02).
- Crear un usuario de base de datos solo para esta app, con permisos únicamente sobre su base, en vez de conectarse como root (A05).
- No mostrarle el error de conexión al usuario, mandarlo a un log (A05 y A09).
- Usar htmlspecialchars al imprimir datos que escribió el usuario, para evitar xss (A03).
- Validar bien las entradas, por ejemplo que la cantidad sea un entero positivo (A04).
- Agregar un token anti-csrf en los formularios (A01).

Discusión general

Si uno mira solo cuántos datos saca de una, la inyección union (la fila 2) parece la peor, porque de un solo envío muestra todos los usuarios con sus contraseñas. Pero la guía pide comparar eso con el ruido que deja el ataquey ahí yo cambio de opinión.

Para mí el mayor riesgo es la inyección ciega booleana del login (fila 3). Al final saca lo mismo que la union, que son las credenciales, pero sin dejar rastro a la vista. No genera errores en pantalla ni salidas raras, solo son intentos de login que, mirados uno por uno, parecen intentos normales de alguien que no se acuerda de la clave. Un atacante puede automatizar eso y sacar la clave carácter por carácter sin que nadie se dé cuenta. Y como Compra Segura no guarda registro de los intentos ni tiene monitoreo (A09), ese ataque podría estar corriendo días y la empresa ni enterarse. En cambio la union y la de error son escandalosas: cualquiera que mire la pantalla o los logs las ve enseguida.

O sea, la union hace más daño rápido pero es fácil de pillary la booleana hace el mismo daño pero calladita. Para el negocio eso último es más grave, porque el robo pasa sin que alcancen a reaccionar.

Los hallazgos además se encadenan: la de error da la versión del motor para preparar el ataque, la union y la booleana sacan las credenciales (que salen legibles porque están en

texto plano)y con esas credenciales o con pedidos sin sesión se puede abusar de registro.php y pedido.php para dañar los datos. Por eso, aunque la booleana es la que más preocupa, la corrección tiene que cubrir los cinco puntos a la vez.

Conclusiones

- Las tres variantes demostradas (error, union y ciega booleana) funcionaron porque buscar_producto.php y login.php pegan lo que escribe el usuario dentro del sql; con la de error se obtuvo la versión del motor y con la union y la booleana, las credenciales (objetivo 1).
- registro.php (línea 6) y pedido.php (línea 10) tienen el mismo error en sus inserty pedido.php además confía en el precio que manda el cliente y acepta pedidos sin sesión, que es un fallo de diseño (objetivo 2).
- La tabla consolidada muestra que los cinco hallazgos vienen de la misma causa y afectan la confidencialidad (filas 1 a 3) y la integridad (filas 4 y 5) (objetivo 3).
- Con consultas preparadas se cierran las cinco inyecciones de una vez, pero hacen falta password_hash, un usuario de base de datos sin privilegios de root y leer el precio desde la base de datos para cerrar los fallos que la consulta preparada no tapa (objetivo 4).
- El mayor riesgo para el negocio es la inyección ciega booleana del login, porque saca las credenciales sin dejar rastro visible y la app no registra ni monitorea los intentos (objetivo 5).